18. useEffect Hook in React

▼ What is the `useEffect` Hook in React?

- The `useEffect()` is one of the basic hook in React.js.
- The `useEffect` hook is used to perform side effects in function-based components.
- OR
- The `useEffect` is a **Hook** that lets you run some code **after React has rendered the UI**.
- **What exactly is an "effect"?**
- The word effect refers to a functional programming term called a "**side effect**".
- **Syntax**:

```
useEffect(() => {
  // Side-effect logic here

  return () => {
    // Cleanup logic here (optional)
  };
}, [dependencies]);
```

- First argument: a **callback function** (effect)
- Optional **cleanup function** returned from inside it
- Second argument: **dependency array**

▼ What are the `side effects` of React?

- We mainly use it for **side effects** like:
  - Fetching an API
  - Working with localStorage / sessionStorage
  - setTimeout / setInterval

- manually working with the DOM (event listeners, focus, etc.)

▼ What do I mean by outside of the React scope?

- It means not part of the React library.
- For Example → The **localStorage** in your browser.
- Local storage is a Web API and not part of the React Universe.
- On a side note, localStorage is an essential tool for building web applications.

▼ Different useEffect Behaviors

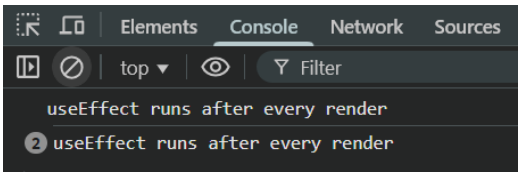
▼ 1. `useEffect()` without dependency array

- It will work as render method if you are not passing any dependency, but side effects should not be in render method.
- **Code:**

```
import React, { useEffect, useState } from "react";

const UseEffect1 = () => {
  let [count, setCount] = useState(0);

  useEffect(() => {
    // runs after every render (initial + re-renders)
    console.log("useEffect runs after every render");
  });
  return (
    <>
      <h1>useEffect() without any dependency</h1>
      <h2>Count: {count}</h2>
      <button
        onClick={() => {
          setCount(count + 1);
        }}
      >
```

```
      Increment
    </button>
    <button
      onClick={() => {
        setCount(count - 1);
      }}
    >
      Decrement
    </button>
    <button onClick={() => setCount(0)}>Reset</button>
  </>
);
};

export default UseEffect1;
```

- **Output:**



▼ 2. Two states changing without dependency

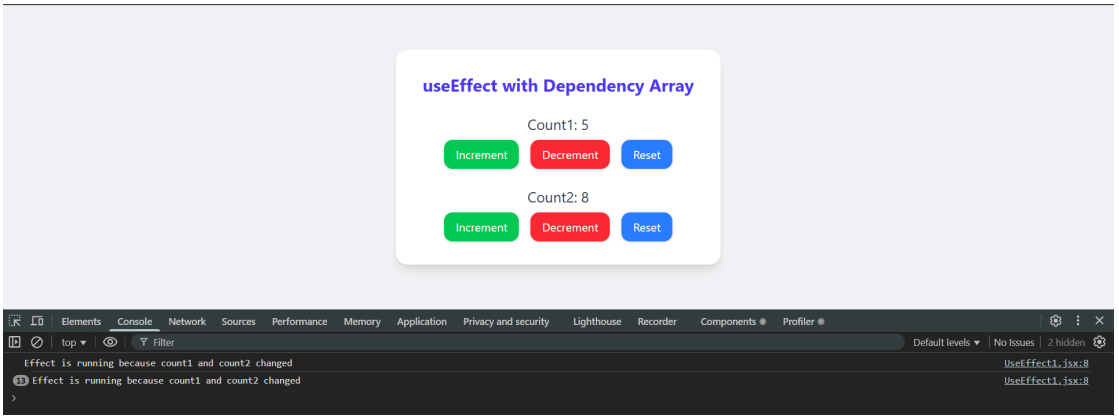
- **Code:**

```
import React, { useEffect, useState } from "react";

const UseEffect1 = () => {
  const [count1, setCount1] = useState(0);
  const [count2, setCount2] = useState(0);

  useEffect(() => {
    console.log("Effect is running because count1 and count2 change
d");
  });

  return (
```

```
<div>
  <h1>
    useEffect with Dependency Array
  </h1>

  {/* Count1 Section */}
  <h2>Count1: {count1}</h2>
  <div>
    <button
      onClick={() => setCount1(count1 + 1)}
      >
      Increment
    </button>
    <button
      onClick={() => setCount1(count1 - 1)}
      >
      Decrement
    </button>
    <button
      onClick={() => setCount1(0)}
      >
      Reset
    </button>
  </div>

  {/* Count2 Section */}
  <h2>Count2: {count2}</h2>
  <div>
    <button
      onClick={() => setCount2(count2 + 1)}
      >
      Increment
    </button>
    <button
      onClick={() => setCount2(count2 - 1)}
      >
      Decrement
    </button>
```

```
        <button
          onClick={() => setCount2(0)}
        >
          Reset
        </button>
      </div>
    </div>
  );
};

export default UseEffect1;
```

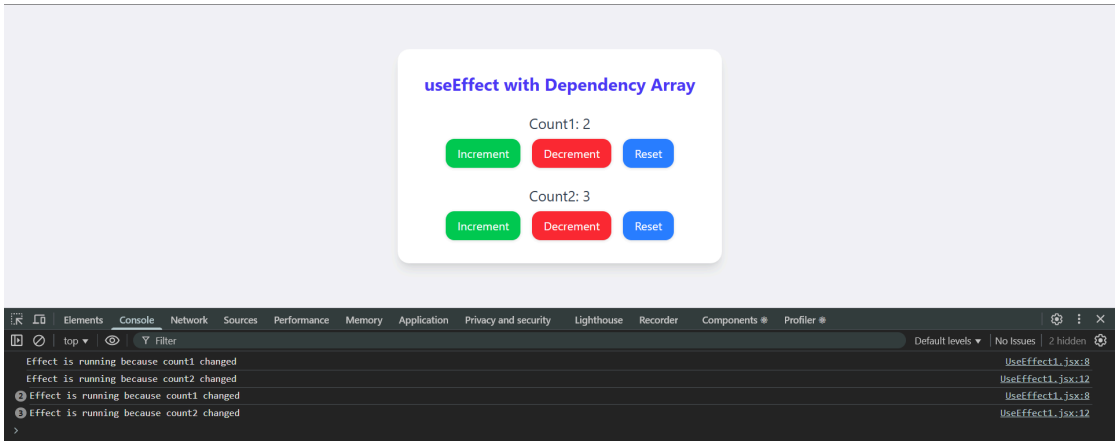
- **Output:**



- **Solution: Provide the dependency array with state(count1 and count2) dependency**
- **Code:**

```
useEffect(() => {
  console.log("Effect is running because count1 changed");
}, [count1]);

useEffect(() => {
  console.log("Effect is running because count2 changed");
}, [count2]);
```

- **Output:**



▼ 3. `useEffect()` with empty dependency array `[]`

- It is equivalent to → `componentDidMount()` in class-based component.
- Fetch data once when the component mounts.
- **Code:**

```
import React, { useEffect, useState } from "react";

const UseEffect1 = () => {
  let [count, setCount] = useState(0);

  useEffect(() => {
    // runs only once, after first render
    console.log("runs only once, after the first render");
  }, []);

  return (
    <>
      <h1>useEffect() with empty dependency array []</h1>
      <h2>Count: {count}</h2>
      <button
        onClick={() => {
          setCount(count + 1);
        }}
      >
        Increment
      </button>
      <button
```

```
      onClick={() => {
        setCount(count - 1);
      }}
    >
      Decrement
    </button>
    <button onClick={() => setCount(0)}>Reset</button>
  </>
);
};

export default UseEffect1
```

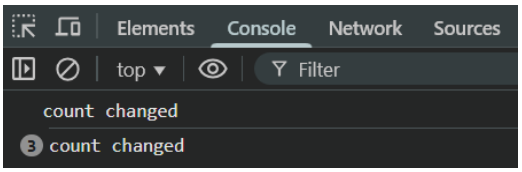
▼ 4. `useEffect()` with a specific state or prop in the dependency array

- Initially it will render the first time.
- But later only when the count changes then only it will render the component.
- It is equivalent to → `componentDidUpdate()` in class-based components. (for that specific dependency)
- Run logic **only when a certain state or prop changes**
- **Code:**

```
import React, { useEffect, useState } from "react";

const UseEffect1 = () => {
  let [count, setCount] = useState(0);

  useEffect(() => {
    // runs when 'count' changes
    console.log("count changed");
  }, [count]);

  return (
    <>
```

```
      <h1>useEffect() with a specific state or prop in the dependency a
rray</h1>
      <h2>Count: {count}</h2>
      <button
        onClick={() => {
          setCount(count + 1);
        }}
      >
        Increment
      </button>
      <button
        onClick={() => {
          setCount(count - 1);
        }}
      >
        Decrement
      </button>
      <button onClick={() => setCount(0)}>Reset</button>
    </>
  );
};

export default UseEffect1;
```

- **Output:**



▼ 5. Returning a function (Cleanup Function)

- **Code: App.jsx**

```
import React, { useState } from "react";
import UseEffect1 from "./UseEffect Hook/UseEffect1";

const App = () => {
```

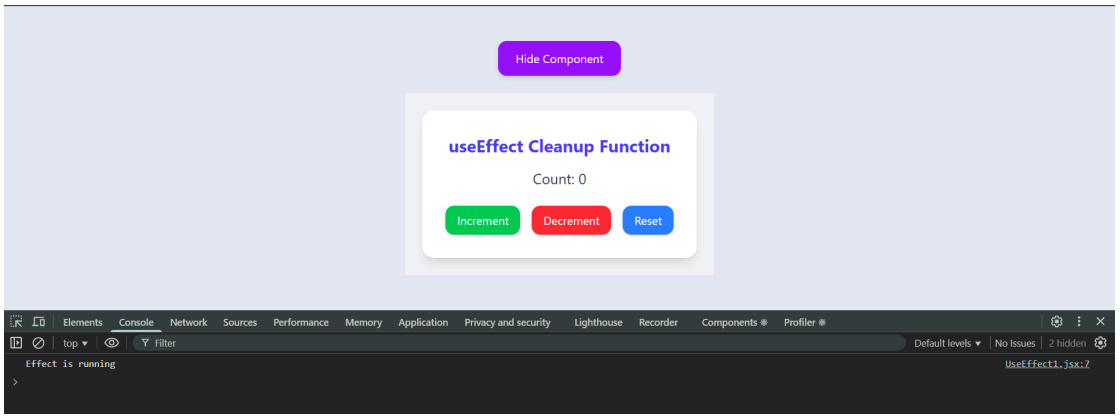
```
  const [show, setShow] = useState(true);

  return (
    <div>
      <button
        onClick={() => setShow(!show)}
        >
        {show ? "Hide Component" : "Show Component"}
      </button>

      {show && <UseEffect1 />}
    </div>
  );
};

export default App;
```

- **Code: UseEffect1.jsx**

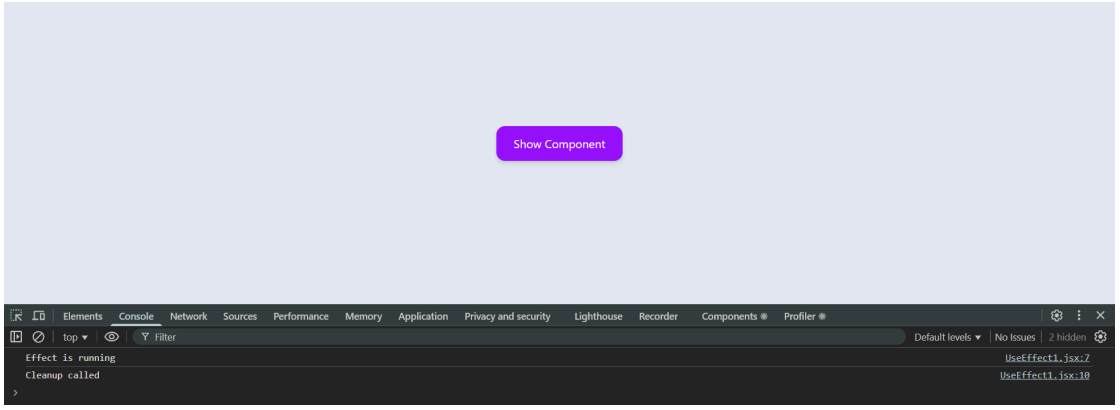
```
import React, { useEffect, useState } from "react";

const UseEffect1 = () => {
  let [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Effect is running");

    return () => {
      console.log("Cleanup called");
    };
  }, []);

  return (
      <div>
        <h1>
          useEffect Cleanup Function
        </h1>
        <h2>Count: {count}</h2>
```

```
      <div>
        <button
          onClick={() => setCount(count + 1)}
          >
          Increment
        </button>
        <button
          onClick={() => setCount(count - 1)}
          >
          Decrement
        </button>
        <button
          onClick={() => setCount(0)}
          >
          Reset
        </button>
      </div>
    </div>
  );
};

export default UseEffect1;
```

- **Output:**



- **After clicking the hide component:**



▼ What is `Cleanup Function`?

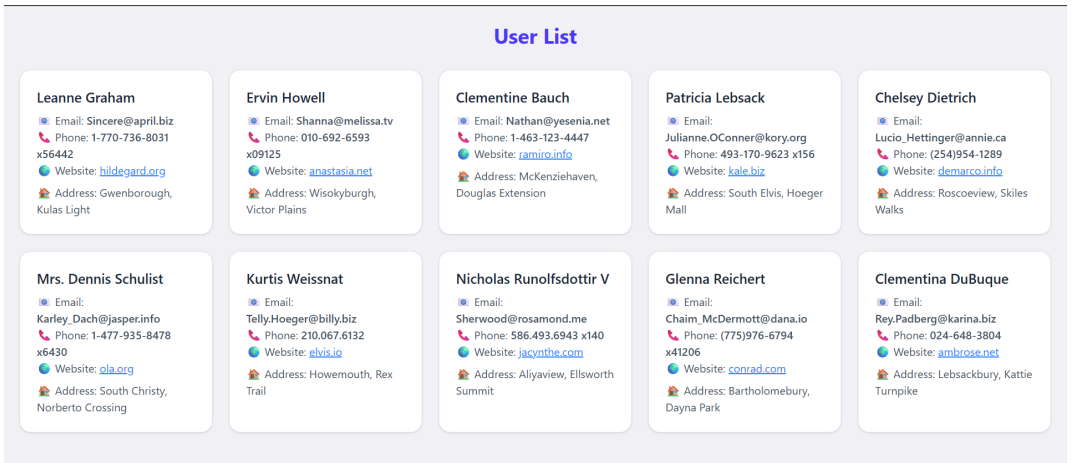
- A **cleanup function** is the **function returned** from inside the `useEffect` callback.
- It is **executed just before** the component **unmounts**.
- It is typically used to **clean up side effects** like:
  - Removing event listeners
  - Clearing timers or intervals
  - Canceling API requests
  - Unsubscribing from services (like WebSocket)

▼ Fetching Data with `async/await` in `useEffect`

- **Code:**

```
import React, { useEffect, useState } from "react";

const FetchUsers = () => {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        );
```

```
      const data = await response.json();
      setUsers(data);
      setLoading(false);
    } catch (error) {
      console.error("Error fetching users:", error);
    } finally {
      setLoading(false);
    }
  };

  fetchData();
}, []);

return (
  <div className="min-h-screen bg-gray-100 p-6 flex flex-col items-c
enter">
    <h1 className="text-3xl font-bold text-indigo-600 mb-8">User List
</h1>

    {loading ? (
      <p className="text-lg text-gray-600">Loading users...</p>
    ) : (
      <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 l
g:grid-cols-5 gap-6 w-full max-w-8xl">
        {users.map((user) => (
          <div
            key={user.id}
            className="bg-white rounded-2xl shadow p-6 border border-
gray-200"
          >
            <h2 className="text-xl font-semibold text-gray-800 mb-2">
              {user.name}
            </h2>
            <p className="text-gray-600">
              📧 Email: <span className="font-medium">{user.email}</sp
an>
            </p>
            <p className="text-gray-600">
```

```
            📞 Phone: <span className="font-medium">{user.phone}</span>
          </p>
          <p className="text-gray-600">
            🌍 Website:{" "}
            <a
              href={`https://${user.website}`}
              className="text-blue-500 underline"
              target="_blank"
              rel="noopener noreferrer"
            >
              {user.website}
            </a>
          </p>
          <p className="text-gray-600 mt-2">
            🏠 Address: {user.address.city}, {user.address.street}
          </p>
        </div>
      ))}
    </div>
  )}
</div>
);
};

export default FetchUsers;
```

- **Output:**